

```

#include <iostream>
#include <vector>
#include <queue>
#include <cmath>
#include <limits>
#include <memory>
#include <algorithm> // Add this line for reverse function

using namespace std;

// Grid size and obstacles are user-defined in main
const int GRID_SIZE = 10; // Grid size (e.g., 10x10)

// Grid to represent the game board
vector<vector<int>> grid(GRID_SIZE, vector<int>(GRID_SIZE, 0));

// Structure for a cell in the grid
struct Cell {
    int x, y;
    double f, g, h;
    shared_ptr<Cell> parent;

    Cell(int x, int y, double g = 0, double h = 0, shared_ptr<Cell> parent
= nullptr)
        : x(x), y(y), g(g), h(h), f(g + h), parent(parent) {}
};

// Heuristic function (Manhattan distance)
double heuristic(int x1, int y1, int x2, int y2) {
    return abs(x1 - x2) + abs(y1 - y2);
}

// A* Algorithm function to find the shortest path
bool aStar(int start_x, int start_y, int goal_x, int goal_y) {
    if (grid[start_x][start_y] == 1 || grid[goal_x][goal_y] == 1) {
        cout << "Start or goal is an obstacle." << endl;
        return false;
    }

    // Priority queue with custom comparator based on f value

```

```

    auto cmp = [](shared_ptr<Cell> a, shared_ptr<Cell> b) { return a->f >
b->f; };
    priority_queue<shared_ptr<Cell>, vector<shared_ptr<Cell>>,
decltype(cmp)> openList(cmp);
    vector<vector<bool>> closedList(GRID_SIZE, vector<bool>(GRID_SIZE,
false));

    // Initialize the start cell
    shared_ptr<Cell> start = make_shared<Cell>(start_x, start_y, 0,
heuristic(start_x, start_y, goal_x, goal_y));
    openList.push(start);

    // Define possible movements (4 directions)
    int dx[] = {-1, 1, 0, 0};
    int dy[] = {0, 0, -1, 1};

    // To store and display the path
    vector<pair<int, int>> path;

    while (!openList.empty()) {
        shared_ptr<Cell> current = openList.top();
        openList.pop();

        // If the goal is reached, reconstruct the path
        if (current->x == goal_x && current->y == goal_y) {
            cout << "Path found: ";
            while (current) {
                path.push_back({current->x, current->y});
                current = current->parent;
            }
            reverse(path.begin(), path.end()); // Reverse to get
start-to-goal order
            for (const auto& coord : path) {
                cout << "(" << coord.first << ", " << coord.second << "
";
            }
            cout << endl;
            return true;
        }
    }

```

```

        // Mark the current cell as closed
        closedList[current->x][current->y] = true;

        // Explore neighbors
        for (int i = 0; i < 4; ++i) {
            int nx = current->x + dx[i];
            int ny = current->y + dy[i];

            // Check if within grid bounds, walkable, and not already
visited
            if (nx >= 0 && nx < GRID_SIZE && ny >= 0 && ny < GRID_SIZE &&
grid[nx][ny] == 0 && !closedList[nx][ny]) {
                double gNew = current->g + 1; // Distance from start to
neighbor
                double hNew = heuristic(nx, ny, goal_x, goal_y);
                openList.push(make_shared<Cell>(nx, ny, gNew, hNew,
current));
            }
        }

        cout << "No path found." << endl;
        return false;
    }

int main() {
    // Initialize grid with 0 (walkable) and 1 (obstacle)
    int num_obstacles;
    cout << "Enter number of obstacles: ";
    cin >> num_obstacles;

    cout << "Enter obstacle positions (x y): " << endl;
    for (int i = 0; i < num_obstacles; ++i) {
        int x, y;
        cin >> x >> y;
        if (x >= 0 && x < GRID_SIZE && y >= 0 && y < GRID_SIZE) {
            grid[x][y] = 1;
        }
    }

    int start_x, start_y, goal_x, goal_y;

```

```

    cout << "Enter start position (x y): ";
    cin >> start_x >> start_y;
    cout << "Enter goal position (x y): ";
    cin >> goal_x >> goal_y;

    // Run A* algorithm
    if (!aStar(start_x, start_y, goal_x, goal_y)) {
        cout << "Path not found." << endl;
    }

    return 0;
}

```

Output:

Enter number of obstacles: 5

Enter obstacle positions (x y):

1 1

1 2

2 1

3 3

4 4

Enter start position (x y): 0 0

Enter goal position (x y): 4 4